

COMP 451 – Compiler Construction

Activity Sheet – Three-Address Code (TAC)

Name: ____Musfira(h) Sehar____ Student ID: ____280048443____ Date: ____14/05/2026____

1. What is Three-Address Code?

Three-Address Code (TAC) is an intermediate representation used by compilers between the high-level source language and machine code. Each TAC instruction has **at most three operands** and performs exactly one operation. This makes it easy to optimize and translate to assembly.

General form: result = operand1 op operand2

Key Rules for Writing TAC:

1. Each instruction does ONE operation (no chained expressions).
2. Intermediate values are stored in temporaries: t1, t2, t3, ...
3. Control flow uses labels (L1:, L2:) and goto / if-goto statements.
4. Array accesses use offset notation: x[i] or x[t1].
5. Boolean expressions are reduced to conditional jumps.

2. Types of TAC Instructions

Type	Template	Example
Assignment	x = y	a = b
Arithmetic	x = y op z	t1 = a + b
Unary	x = op y	t1 = -a
Copy	x = y	x = t1
Unconditional Jump	goto L	goto L3
Conditional Jump	if x goto L	if t1 > 0 goto L2
Relational	if x relop y goto L	if a < b goto L1
Param / Call	param x / call p, n	param a / call f, 2
Return	return x	return t2
Array Read	x = y[i]	t1 = a[i]
Array Write	x[i] = y	a[i] = t1
Pointer Deref	x = *y / *x = y	x = *p

3. Arithmetic Expressions & Assignments

Complex expressions must be broken into a sequence of simple assignments, one operator at a time.

Worked Example 3.1 – Arithmetic Expression

C Code	Three-Address Code
--------	--------------------

// C Code	// TAC
x = (a + b) * (c - d);	t1 = a + b
	t2 = c - d
	x = t1 * t2

Worked Example 3.2 – Multiple Assignments

C Code	Three-Address Code
// C Code	// TAC
int a = 5;	a = 5
int b = a * 2 + 3;	t1 = a * 2
int c = b - a;	t2 = t1 + 3
	b = t2
	c = b - a

Exercise 1: Arithmetic Expression

Convert the following C code to Three-Address Code. Use temporaries t1, t2, t3, ... as needed.

```
x = (a * b) + (c / d) - e;
```

Your TAC:

// C Code:	//TAC
x=(a * b) + (c / d) - e	t1 = a * b
	t2 = c / d
	t3 = t1 + t2
	t4 = e
	t5 = t3 - t4

Exercise 2: Nested Expression

Convert the following C code to Three-Address Code. Use temporaries t1, t2, t3, ... as needed.

```
y = (p + q) * (p - q);
z = y / 2 + p;
```

Your TAC:

// C Code:	//TAC
y = (p + q) * (p - q)	t1 = p + q
	t2 = p - q
	t3 = t1 * t2
z = y / 2 + p	t1 = y
	t2 = 2 + p

t3 = t1 / t2

4. If-Else Statements

Conditional statements are translated using **if-goto** and **goto** instructions with labels.

Template for if-else:

```
if NOT (condition) goto Lelse
<then-body TAC>
goto Lend
Lelse:
<else-body TAC>
Lend:
```

Worked Example 4.1 – Simple If

C Code	Three-Address Code
// C Code	// TAC
if (a > b) {	if a <= b goto L1
x = a;	x = a
}	L1:

Worked Example 4.2 – If-Else

C Code	Three-Address Code
// C Code	// TAC
if (x < 10) {	if x >= 10 goto Lelse
y = x + 1;	t1 = x + 1
} else {	y = t1
y = x - 1;	goto Lend
}	Lelse:
	t2 = x - 1
	y = t2
	Lend:

Worked Example 4.3 – Nested If-Else

C Code	Three-Address Code
// C Code	// TAC
if (a > 0) {	if a <= 0 goto L3
if (b > 0) {	if b <= 0 goto L2
z = a + b;	t1 = a + b
} else {	z = t1
z = a - b;	goto L1
}	L2:

} else {	t2 = a - b
z = 0;	z = t2
}	goto L1
	L3:
	z = 0
	L1:

Exercise 3: If-Else Statement

Convert the following C code to Three-Address Code. Use temporaries t1, t2, t3, ... as needed.

```
if (score >= 50) {
    grade = 1;
} else {
    grade = 0;
}
```

Your TAC:

// C Code	// TAC
if (score >= 50) { grade = 1; } else { grade = 0; }	if score <= 50 goto Lelse t1 = 1 grade = t1 goto Lend Lelse: t2 = 0 grade = t2 Lend

Exercise 4: Nested If-Else

Convert the following C code to Three-Address Code. Use temporaries t1, t2, t3, ... as needed.

```
if (x > 0) {
    y = x * 2;
} else if (x < 0) {
    y = -x;
} else {
    y = 0;
}
```

Your TAC:

// C Code	// TAC
if (x > 0) { y = x * 2; } else if (x < 0) { y = -x;	if x <= 0 goto L1 t1 = x * 2 y = t1 goto Lend if x > 0 goto L2 t2 = -x y = t2

<pre> } else { y = 0; } </pre>	<pre> goto Lend Lelse t3 = 0 y = t3 goto Lend </pre>
------------------------------------	--

5. While Loops

A **while** loop tests the condition before each iteration. The test is placed at the top of the TAC loop.

Template for while:

Lstart:

```

if NOT (condition) goto Lend
<loop-body TAC>
goto Lstart

```

Lend:

Worked Example 5.1 – While Loop

C Code	Three-Address Code
// C Code	// TAC
int i = 0;	i = 0
while (i < n) {	L1:
sum = sum + i;	if i >= n goto L2
i = i + 1;	t1 = sum + i
}	sum = t1
	t2 = i + 1
	i = t2
	goto L1
	L2:

Exercise 5: While Loop

Convert the following C code to Three-Address Code. Use temporaries t1, t2, t3, ... as needed.

```

int i = 1;
int fact = 1;
while (i <= n) {
    fact = fact * i;
    i = i + 1;
}

```

Your TAC:

// C Code

// TAC

```

int i = 1;
int fact = 1;
while (i <= n) {
    fact = fact * i;

    i = i + 1;

}

```

```

i = 1
fact = 1
L1:
t1 = fact * i
fact = t1
t2 = i + 1
i = t2
goto L1
L2:

```

6. Do-While Loops

A **do-while** loop executes the body at least once. The condition is checked at the **bottom** of the TAC loop.

Template for do-while:

Lstart:

<loop-body TAC>

if (condition) goto Lstart

Lend:

Worked Example 6.1 – Do-While Loop

C Code	Three-Address Code
// C Code	// TAC
int i = 0;	i = 0
do {	L1:
x = x + i;	t1 = x + i
i = i + 1;	x = t1
} while (i < 5);	t2 = i + 1
	i = t2
	if i < 5 goto L1
	L2:

Exercise 6: Do-While Loop

Convert the following C code to Three-Address Code. Use temporaries t1, t2, t3, ... as needed.

```

int count = 0;
do {
    count = count + 1;
} while (count < 10);

```

Your TAC:

// C Code

```
int count = 0;
```

// TAC

```
count = 0
```

do {	L1:
count = count + 1;	t1 = count + 1
	count = t1
} while (count < 10);	if count < 10 goto L1
	L2

7. For Loops

A **for** loop has three parts: initialization, condition, and update. Each part maps directly to TAC.

Template for for-loop:

```

<init TAC>
Lstart:
  if NOT (condition) goto Lend
  <body TAC>
  <update TAC>
  goto Lstart
Lend:

```

Worked Example 7.1 – For Loop

C Code	Three-Address Code
// C Code	// TAC
for (i = 0; i < n; i++) {	i = 0
sum = sum + a[i];	L1:
}	if i >= n goto L2
	t1 = i * 4
	t2 = a[t1]
	t3 = sum + t2
	sum = t3
	t4 = i + 1
	i = t4
	goto L1
	L2:

Exercise 7: For Loop

Convert the following C code to Three-Address Code. Use temporaries t1, t2, t3, ... as needed.

```

int product = 1;
for (int i = 1; i <= n; i++) {
    product = product * i;
}

```

Your TAC:

//TAC

product = 1

i = 1

L1:

if i > n goto L2

t1 = product * i

product = t1

t2 = i + 1

i = t2

goto L1

L2:

// C Code (int a[N])

int product = 1;

for (int i = 1; i <= n; i++) {

product = product * i;

}

8. Boolean Expressions

Boolean expressions use && (AND), || (OR), and ! (NOT). In TAC, they are converted to conditional jumps using **short-circuit evaluation**.

Short-Circuit Rules:

For (E1 && E2): if E1 is false, skip E2 (jump to false branch)

For (E1 || E2): if E1 is true, skip E2 (jump to true branch)

For (!E): negate the condition of E

Worked Example 8.1 – AND Condition

C Code	Three-Address Code
// C Code	// TAC
if (a > 0 && b > 0) {	if a <= 0 goto L1
x = a + b;	if b <= 0 goto L1
}	t1 = a + b
	x = t1
	L1:

Worked Example 8.2 – OR Condition

C Code	Three-Address Code
--------	--------------------

// C Code	// TAC
if (a == 0 b == 0) {	if a == 0 goto L1
x = 0;	if b != 0 goto L2
}	L1:
	x = 0
	L2:

Worked Example 8.3 – NOT Condition

C Code	Three-Address Code
// C Code	// TAC
if (!(x > 5)) {	if x > 5 goto L1
y = 0;	y = 0
}	L1:

Exercise 8: Boolean AND / OR

Convert the following C code to Three-Address Code. Use temporaries t1, t2, t3, ... as needed.

```
if (x >= 0 && x <= 100) {
    valid = 1;
} else {
    valid = 0;
}
```

Your TAC:

//TAC

```
if x < 0 goto Lelse
if x > 100 goto Lelse
valid = 1
goto Lend
Lelse:
valid = 0
Lend:
```

// C Code (int a[N])

```
if (x >= 0 && x <= 100) {
    valid = 1;
} else {
    valid = 0;
}
```

Exercise 9: Boolean in While Loop

Convert the following C code to Three-Address Code. Use temporaries t1, t2, t3, ... as needed.

```
while (i < n && a[i] != 0) {
    sum = sum + a[i];
    i = i + 1;
}
```

Your TAC:

//TAC	// C Code (int a[N])
L1:	while (i < n && a[i] != 0) {
if i >= n goto L2	
t1 = i * 4	
t2 = a[t1]	
if t2 == 0 goto L2	
t3 = sum + t2	sum = sum + a[i];
sum = t3	
t4 = i + 1	i = i + 1;
i = t4	
goto L1	
L2:	}

9. One-Dimensional Arrays

9.1 How Array Addressing Works in TAC

Element address formula: $\text{address}(a[i]) = \text{base}(a) + i \times \text{width}$

- $\text{base}(a)$ is the starting address of the array (often written simply as a).
- width is the size of each element in bytes (e.g., 4 bytes for `int`).
- For `int a[N]`: $a[i] \rightarrow \text{offset} = i * 4$

TAC Templates for 1-D Arrays:

Read: $x = a[i]$
 $t1 = i * \text{width}$ (compute byte offset)
 $x = a[t1]$ (load value at offset)

Write: $a[i] = x$
 $t1 = i * \text{width}$ (compute byte offset)
 $a[t1] = x$ (store value at offset)

Worked Example 9.1 – Reading from 1-D Array

C Code	Three-Address Code
// C Code (int a[N])	// TAC
$x = a[i];$	$t1 = i * 4$
	$x = a[t1]$

Worked Example 9.2 – Writing to 1-D Array

C Code	Three-Address Code
// C Code	// TAC
a[i] = x + 1;	t1 = x + 1
	t2 = i * 4
	a[t2] = t1

Worked Example 9.3 – Loop Over 1-D Array

C Code	Three-Address Code
// C Code	// TAC
for (i=0; i<n; i++) {	i = 0
b[i] = a[i] * 2;	L1:
}	if i >= n goto L2
	t1 = i * 4
	t2 = a[t1] // read a[i]
	t3 = t2 * 2
	t4 = i * 4
	b[t4] = t3 // write b[i]
	t5 = i + 1
	i = t5
	goto L1
	L2:

Exercise 10: 1-D Array Read in a Loop

Convert the following C code to Three-Address Code. Use temporaries t1, t2, t3, ... as needed.

```
// Assume int a[N], int sum = 0
for (int i = 0; i < n; i++) {
    sum = sum + a[i];
}
```

Your TAC:

//TAC

sum = 0

i = 0

L1:

if i >= n goto L2

t1 = i * 4

t2 = a[t1]

t3 = sum + t2

// C Code (int a[N])

int sum = 0

for (int i = 0; i < n; i++) {

sum = sum + a[i];

```

sum = t3
t4 = i + 1
i = t4
goto L1
L2:

```

Exercise 11: 1-D Array Write

Convert the following C code to Three-Address Code. Use temporaries t1, t2, t3, ... as needed.

```

// Assume int b[N]
for (int i = 0; i < n; i++) {
    b[i] = i * i;
}

```

Your TAC:

<pre> //TAC i = 0 L1: if i >= n goto L2 t1 = i * i t2 = i * 4 b[t2] = t1 t3 = i + 1 i = t3 goto L1 L2: </pre>	<pre> // C Code (int a[N]) for (int i = 0; i < n; i++) { } </pre>
--	--

10. Two-Dimensional Arrays

10.1 Row-Major Order Addressing

Element address formula (row-major): $\text{address}(a[i][j]) = \text{base}(a) + (i \times \text{ncols} + j) \times \text{width}$

- ncols is the number of columns (second dimension size).
- width is the size of each element in bytes.
- C stores 2-D arrays in **row-major order**: all elements of row 0 first, then row 1, etc.

TAC Template for 2-D Array Access:

Read: `x = a[i][j]` (array has `ncols` columns, `int` elements)

`t1 = i * ncols` (row offset in elements)

`t2 = t1 + j` (element index)

`t3 = t2 * 4` (byte offset)

`x = a[t3]` (load value)

Write: `a[i][j] = x`

`t1 = i * ncols`

`t2 = t1 + j`

`t3 = t2 * 4`

`a[t3] = x`

Worked Example 10.1 – Reading from 2-D Array

C Code	Three-Address Code
// C Code	// TAC (ncols = 4)
// int a[M][N], N = 4	<code>t1 = i * 4</code> // <code>i * ncols</code>
<code>x = a[i][j];</code>	<code>t2 = t1 + j</code>
	<code>t3 = t2 * 4</code> // byte offset
	<code>x = a[t3]</code>

Worked Example 10.2 – Writing to 2-D Array

C Code	Three-Address Code
// C Code	// TAC
// int a[M][N], N = 4	<code>t1 = x + y</code>
<code>a[i][j] = x + y;</code>	<code>t2 = i * 4</code>
	<code>t3 = t2 + j</code>
	<code>t4 = t3 * 4</code>
	<code>a[t4] = t1</code>

Worked Example 10.3 – Nested Loop over 2-D Array

C Code	Three-Address Code
// C Code	// TAC
// int a[M][N], N cols	<code>i = 0</code>
for (i=0; i<M; i++) {	L1: if <code>i >= M</code> goto L4
for (j=0; j<N; j++) {	<code>j = 0</code>
<code>a[i][j] = 0;</code>	L2: if <code>j >= N</code> goto L3
}	<code>t1 = i * N</code>
}	<code>t2 = t1 + j</code>
	<code>t3 = t2 * 4</code>
	<code>a[t3] = 0</code>
	<code>t4 = j + 1</code>
	<code>j = t4</code>

	goto L2
	L3: t5 = i + 1
	i = t5
	goto L1
	L4:

Exercise 12: 2-D Array – Single Element Access

Convert the following C code to Three-Address Code. Use temporaries t1, t2, t3, ... as needed.

```
// int mat[4][4]
// Read mat[r][c] and double it
mat[r][c] = mat[r][c] * 2;
```

Your TAC:

//TAC

t1 = r * 4

t2 = t1 + c

t3 = t2 * 4

t4 = mat[t3]

t5 = t4 * 2

mat[t3] = t5

// C Code (int a[N])

mat[r][c] = mat[r][c] * 2;

Exercise 13: 2-D Array – Row Sum

Convert the following C code to Three-Address Code. Use temporaries t1, t2, t3, ... as needed.

```
// int a[M][N]
int rowSum = 0;
for (int j = 0; j < N; j++) {
    rowSum = rowSum + a[i][j];
}
```

Your TAC:

//TAC

rowSum = 0

j = 0

L1:

if j >= N goto L2

t1 = i * N

t2 = t1 + j

t3 = t2 * 4

t4 = a[t3]

// C Code (int a[N])

int rowSum = 0;

for (int j = 0; j < N; j++) {

rowSum = rowSum + a[i][j];

```

t5 = rowSum + t4
rowSum = t5
t6 = j + 1
j = t6
goto L1
L2:                                }

```

11. Quick Reference Summary

Construct	Key TAC Pattern
if (cond)	if NOT cond goto Lend / ...body... / Lend:
if-else	if NOT cond goto Lelse / ...then... / goto Lend / Lelse: / ...else... / Lend:
while	Lstart: / if NOT cond goto Lend / ...body... / goto Lstart / Lend:
do-while	Lstart: / ...body... / if cond goto Lstart
for	init / Lstart: / if NOT cond goto Lend / body / update / goto Lstart / Lend:
E1 && E2	if NOT E1 goto Lfalse / if NOT E2 goto Lfalse
E1 E2	if E1 goto Ltrue / if NOT E2 goto Lfalse
a[i] (read)	t1 = i * width / x = a[t1]
a[i] (write)	t1 = i * width / a[t1] = x
a[i][j] (read)	t1 = i*ncols+j / t2 = t1*width / x = a[t2]
a[i][j] (write)	t1 = i*ncols+j / t2 = t1*width / a[t2] = x